

[WS09] Update and Delete Data

308-493 CROSS-PLATFORM MOBILE PROGRAMMING AND APPLICATIONS

สิ่งที่จะได้เรียนจาก workshop นี้

การส่งและใช้ Custom Callback

การสร้าง Confirm dialog

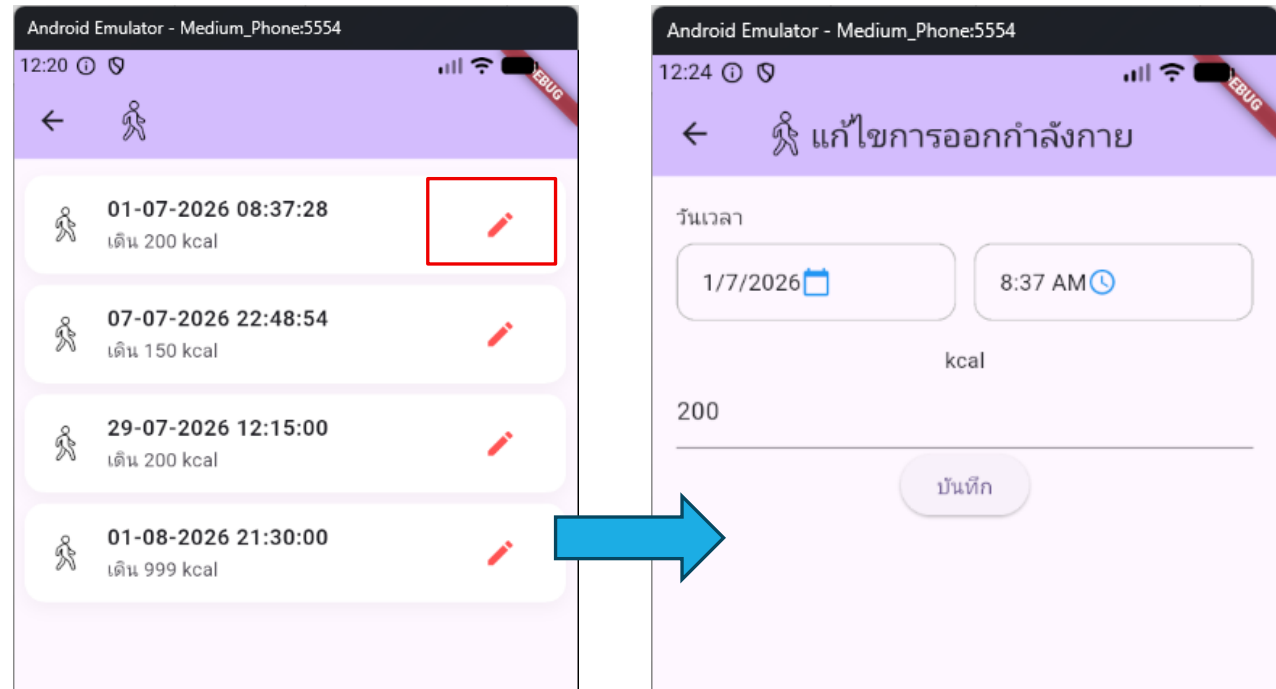
[WS09-1] ดึงข้อมูลของ activity เพื่อเตรียม
แก้ไข

ดึงข้อมูล activity เดิมมาแสดงใน screen

เมื่อผู้ใช้กดปุ่มแก้ไข เราจะ
เปลี่ยนหน้าไปยัง
ActivityScreen (จาก A ไป
B) โดยส่งข้อมูล activityId
ไป

เมื่อ activityId ไม่เท่ากับ 0
แปลว่า ผู้ใช้ต้องการแก้ไข
ข้อมูล activity เดิม

เราจะดึงข้อมูลเดิมจาก
ฐานข้อมูลมาแสดงใน
screen เพื่อเตรียมให้ผู้ใช้
แก้ไขข้อมูล



A

B

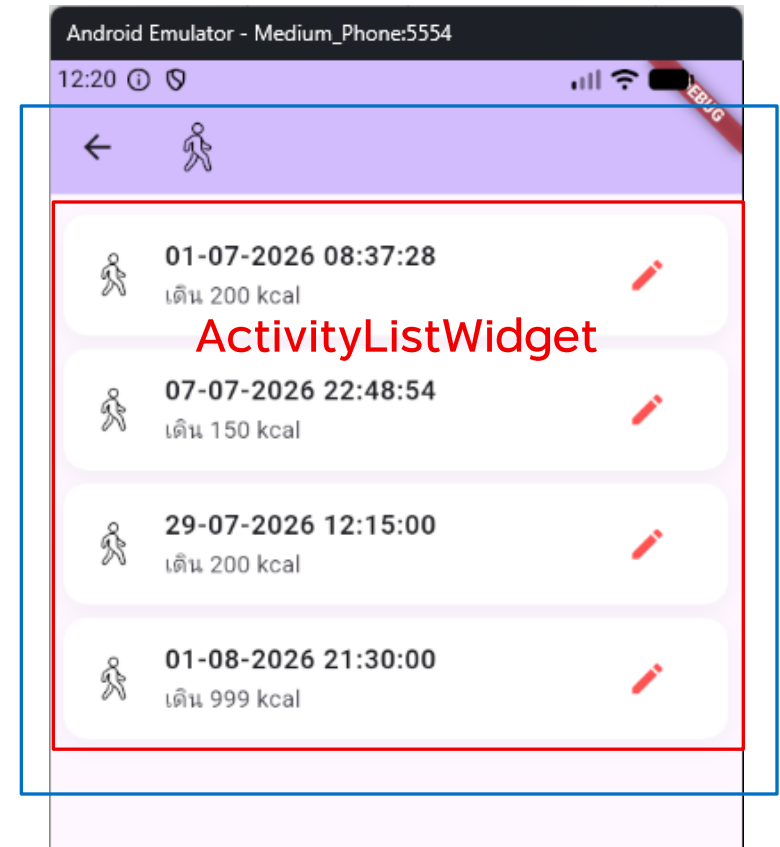
ดึงข้อมูล activity เดิมมาแสดงใน screen

ActivityListWidget เป็นลูก
ของ ActivityListScreen

แต่การสั่ง load ข้อมูลอยู่ใน
ActivityListScreen (ใน
_fetchData)

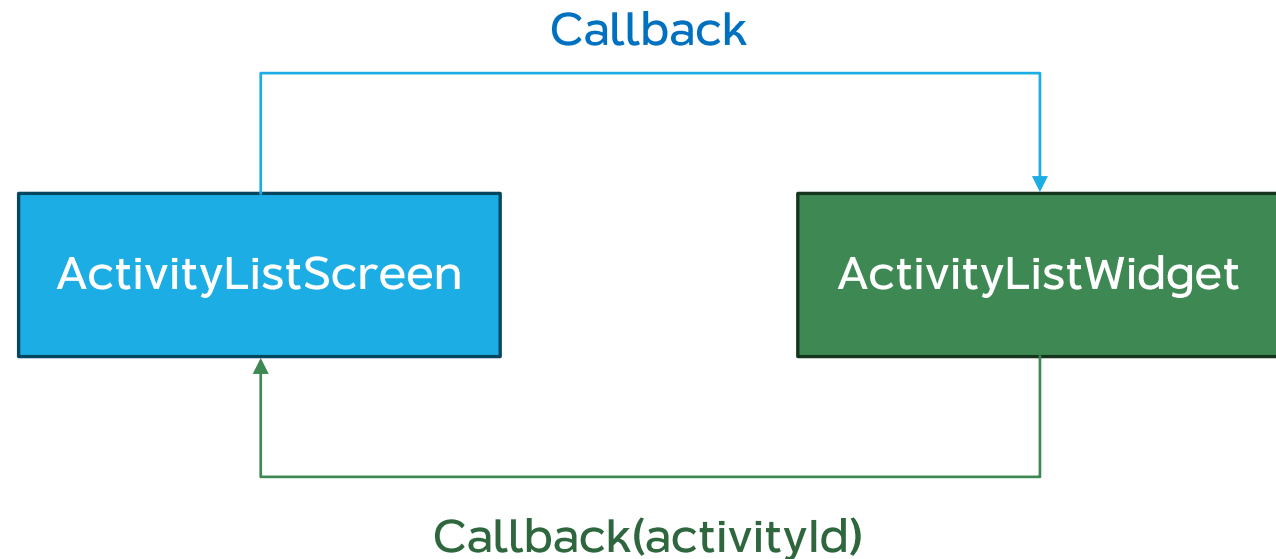
ActivityListScreen

```
class _ActivityListScreenState extends State<ActivityListScreen> {  
  List<ActivityModel> store = [];  
  
  void _fetchData() async {  
    var response = await AppAPI.get(  
      "/activities/get_by_exercise/${widget.exerciseId}",  
    );  
    Map<String, dynamic> json = jsonDecode(response.body);  
    ActivityResponse activityResponse = ActivityResponse.fromJson(json);  
  
    setState(() {  
      store = activityResponse.data;  
    });  
  }  
}
```



ดึงข้อมูล activity เดิมมาแสดงใน screen

ดังนั้นเราจะใช้วิธีการสร้าง Custom Callback เพื่อ กำหนดให้ปุ่ม "แก้ไข" ใน ActivityListWidget ส่ง ข้อมูล activityId กลับมาให้ ActivityListScreen เพื่อ นำไปยังหน้าแก้ไขต่อไป



ดึงข้อมูล activity เดิมมาแสดงใน screen

จากนั้นเราจะส่ง activityId ไปยัง ActivityScreen เพื่อ load ข้อมูลต่อไป



สร้าง Backend สำหรับดึงข้อมูล Activity

/models/activities.js

สร้าง function ชื่อ getActivityById ใน activities.js

เราจะใช้ function นี้สำหรับดึงข้อมูลเดิมของ activity ที่เราต้องการแก้ไข

```
getActivityById: async (activityId) => {
  let conn;
  let result;

  try {
    conn = await pool.getConnection();

    var sql = "SELECT a.act_id, a.ex_id, b.ex_name, "
      + "DATE_FORMAT(a.act_date, '%d-%m-%Y %H:%i:%s') AS act_date, "
      + "a.kcal_expense "
      + "FROM activities a "
      + "JOIN exercises b ON a.ex_id = b.ex_id "
      + "WHERE act_id = ? ";

    var rows = await conn.query(sql, [activityId]);

    result = {
      isError: false,
      data: rows,
      errorMessage: ""
    };
  } catch (error) {
    result = {
      isError: false,
      data: "",
      errorMessage: error.message
    };
  } finally {
    if (conn)
      conn.release();

    return result;
  }
},
```


สร้าง Backend สำหรับดึงข้อมูล Activity

/server.js

เพิ่ม endpoint สำหรับดึงข้อมูล activity

```
app.get("/api/activities/:activityId", checkAccessToken, async (req, res) => {  
  const activityId = req.params.activityId;  
  const response = await activities.getActivityById(activityId);  
  res.json(response);  
});
```

แก้ไข widget

[/lib/widgets/activity_list_widget.dart](#)

แก้ไข ActivityListWidget โดยเพิ่ม attribute ชื่อ onEditPressed สำหรับใช้รับ Callback จาก ActivityListScreen

กำหนดให้รับ onEditPressed มาใน constructor

onEditPressed จะรับตัวแปร 1 ตัว เป็น int ซึ่งเราจะใช้ส่ง activityId

```
class ActivityListWidget extends StatelessWidget {  
  const ActivityListWidget({  
    super.key,  
    required this.activityModelStore,  
    required this.onEditPressed,  
  });  
  final List<ActivityModel> activityModelStore;  
  final Function(int) onEditPressed;
```

แก้ไข widget

[/lib/widgets/activity_list_widget.dart](#)

แก้ไข event ชื่อ onPressed ในปุ่ม
แก้ไข

เราจะเรียกใช้ callback ที่ถูก
ส่งผ่านมากับ constructor

```
trailing: IconButton(  
  icon: const Icon(Icons.edit),  
  color: Colors.redAccent,  
  splashColor: Colors.red.withValues(alpha: 0.1),  
  onPressed: () {  
    onEditPressed(item.activityId);  
  },  
), // IconButton
```

แก้ไข screen

[/lib/screens/activity_list_screen.dart](#)

แก้ไข ActivityListScreen ในส่วนที่เรียกใช้งาน ActivityListWidget

เพิ่ม onEditPressed เข้าไปใน constructor

กำหนดให้เมื่อเปลี่ยน screen กลับจะ load ข้อมูลใหม่ทุกครั้ง

```
body: ActivityListWidget(  
  activityModelStore: store,  
  onEditPressed: (actId) async {  
    await Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => ActivityScreen(  
          exerciseId: widget.exerciseId,  
          activityId: actId,  
        ), // ActivityScreen  
      ), // MaterialPageRoute  
    );  
  
    _fetchData();  
  },  
), // ActivityListWidget
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

สร้าง function ชื่อ
_getActivityModel เพื่อดึงข้อมูล
activity เดิมจาก backend

```
Future<ActivityModel> _getActivityModel() async {  
  var response = await AppAPI.get("/activities/${widget.activityId}");  
  Map<String, dynamic> json = jsonDecode(response.body);  
  ActivityResponse activityResponse = ActivityResponse.fromJson(json);  
  
  ActivityModel model = activityResponse.data[0];  
  
  return model;  
}
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

สร้าง function ชื่อ `_initData` เพื่อดึงข้อมูล activity เดิมจาก backend ผ่าน `_getActivityModel`

จากนั้นนำ `_initData` ไปใช้ใน `initState` ทดสอบโปรแกรม

```
Future<void> _initData() async {  
  if (widget.activityId == 0) {  
    setState(() {  
      titleText = "เพิ่มการออกกำลังกาย";  
      _kcalValueController.text = "0";  
    });  
  } else {  
    ActivityModel model = await _getActivityModel();  
  
    setState(() {  
      titleText = "แก้ไขการออกกำลังกาย";  
      _selectedDate = model.getActivityDateTime();  
      _selectedTime = TimeOfDay.fromDateTime(model.getActivityDateTime());  
      kcalExpense = model.kcalExpense;  
      _kcalValueController.text = model.kcalExpense.toString();  
    });  
  }  
}
```

```
@override  
void initState() {  
  super.initState();  
  _initData();  
}
```

[WS09-2] การแก้ไขข้อมูล

สร้าง Backend สำหรับแก้ไขข้อมูล Activity

/models/activities.js

สร้าง function ชื่อ updateActivity ใน activities.js

```
updateActivity: async (activityId, activityDate, kcalExpense) => {
  let conn;
  let result;

  try {
    conn = await pool.getConnection();

    var sql = "UPDATE activities SET "
      + "act_date = STR_TO_DATE(?, '%d-%m-%Y %H:%i:%s'), "
      + "kcal_expense = ? "
      + "WHERE act_id = ?";

    await conn.query(sql, [activityDate, kcalExpense, activityId]);

    result = {
      isError: false,
      data: "",
      errorMessage: ""
    };
  } catch (error) {
    result = {
      isError: true,
      data: "",
      errorMessage: error.message
    };
  } finally {
    if (conn)
      conn.release();
  }

  return result;
},
```


สร้าง Backend สำหรับแก้ไขข้อมูล Activity

/server.js

สร้าง endpoint สำหรับแก้ไขข้อมูล

```
app.post("/api/activities/update", checkAccessToken, async (req, res) => {  
  const activityDate = req.body.activity_date;  
  const kcalExpense = req.body.kcal_expense;  
  const activityId = req.body.activity_id;  
  
  const response = await activities.updateActivity(activityId, activityDate, kcalExpense);  
  res.json(response);  
});
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

สร้าง function ชื่อ
_doUpdateActivity สำหรับแก้ไข
ข้อมูล

```
Future<(bool, String)> _doUpdateActivity() async {
  DateTime activityDate = DateTime(
    _selectedDate.year,
    _selectedDate.month,
    _selectedDate.day,
    _selectedTime.hour,
    _selectedTime.minute,
  );

  Map<String, dynamic> postData = {
    "activity_date": DateFormat('dd-MM-yyyy HH:mm:ss').format(activityDate),
    "kcal_expense": kcalExpense,
    "activity_id": widget.activityId,
  };

  final response = await AppAPI.post("/activities/update", postData);

  final json = jsonDecode(response.body);

  return (json["isError"] as bool, json["errorMessage"] as String);
}
```

แก้ไข ActivityScreen

[/lib/screens/activity_screen.dart](#)

แก้ไขการทำงานของปุ่มบันทึกข้อมูล
โดยแยกการดำเนินการตาม activityId

ถ้า activityId = 0 เราจะเพิ่มข้อมูลโดยใช้ _doCreateActivity

ถ้า activityId ไม่เท่ากับ 0 เราจะแก้ไขข้อมูลโดยใช้ _doUpdateActivity

ทดสอบโปรแกรม

```
ElevatedButton(  
  onPressed: () async {  
    var isError = false;  
    var errorMessage = "";  
  
    if (widget.activityId == 0) {  
      (isError, errorMessage) = await _doCreateActivity();  
    } else {  
      (isError, errorMessage) = await _doUpdateActivity();  
    }  
  
    if (!isError) {  
      Navigator.pop(context);  
    } else {  
      showDialog(  
        context: context,  
        builder: (context) {  
          return AlertDialog(content: Text(errorMessage));  
        },  
      );  
    }  
  },  
),
```

[WS09-3] การลบข้อมูล

สร้าง Backend สำหรับลบข้อมูล Activity

/models/activities.js

สร้าง function ชื่อ deleteActivity ใน activities.js

```
deleteActivity: async (activityId) => {  
  let conn;  
  let result;  
  
  try {  
    conn = await pool.getConnection();  
  
    var sql = "DELETE FROM activities WHERE act_id = ?";  
  
    await conn.query(sql, [activityId]);  
  
    result = {  
      isError: false,  
      data: "",  
      errorMessage: ""  
    };  
  } catch (error) {  
    result = {  
      isError: true,  
      data: "",  
      errorMessage: error.message  
    };  
  } finally {  
    if (conn)  
      conn.release();  
  
    return result;  
  }  
},
```

สร้าง Backend สำหรับลบข้อมูล Activity

[/models/activities.js](#)

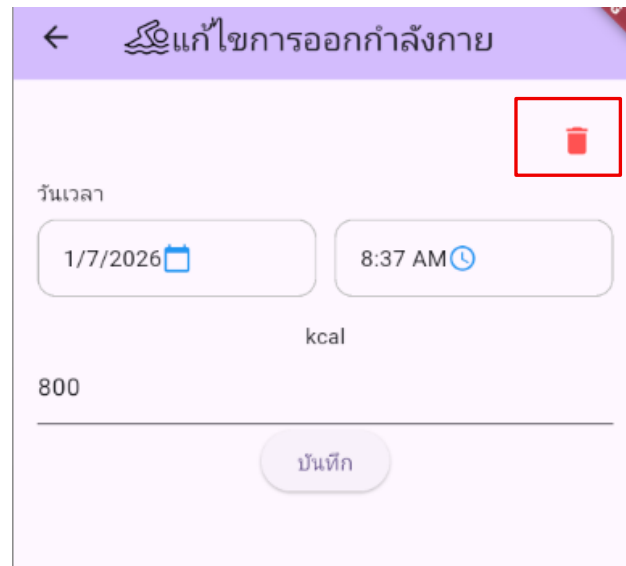
สร้าง endpoint สำหรับลบข้อมูล

```
app.post("/api/activities/delete", checkAccessToken, async (req, res) => {  
  const activityId = req.body.activity_id;  
  
  const response = await activities.deleteActivity(activityId);  
  res.json(response);  
});
```

แก้ไข ActivityScreen

[/lib/screens/activity_screen.dart](#)

สร้างปุ่มสำหรับลบข้อมูลด้านบนของ
DateTimePicker



```
children: [
  Container(
    alignment: Alignment.centerRight,
    child: IconButton(
      icon: const Icon(Icons.delete),
      color: Colors.redAccent,
      splashColor: Colors.red.withValues(alpha: 0.1),
      onPressed: () {
        // Delete button logic
      },
    ), // IconButton
  ), // Container
), // Container
DateTimePickerWidget(
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

สร้าง function ชื่อ
_doDeleteActivity สำหรับลบข้อมูล

```
Future<(bool, String)> _doDeleteActivity() async {  
  Map<String, dynamic> postData = {"activity_id": widget.activityId};  
  
  final response = await AppAPI.post("/activities/delete", postData);  
  
  final json = jsonDecode(response.body);  
  
  return (json["isError"] as bool, json["errorMessage"] as String);  
}
```


แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

สร้าง Confirm Dialog สำหรับให้ผู้ใช้
ยืนยันในการลบข้อมูล

```
Future<void> showConfirmDialog(  
  BuildContext context,  
  Function onConfirm,  
) async {  
  return showDialog<void>(  
    context: context,  
    barrierDismissible: false,  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('ยืนยันการทำการรายการ'),  
        content: const Text('คุณต้องการลบข้อมูลนี้ใช่หรือไม่?'),  
        actions: <Widget>[  
          TextButton(  
            child: const Text('ยกเลิก'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ), // TextButton  
          TextButton(  
            child: const Text('ยืนยัน', style: TextStyle(color: Colors.red)),  
            onPressed: () {  
              Navigator.of(context).pop();  
              onConfirm();  
            },  
          ), // TextButton  
        ], // <Widget>[]  
      ); // AlertDialog  
    },  
  );  
}
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

ใส่ callback ชื่อ onConfirm สำหรับ
ดำเนินการลบข้อมูลเมื่อผู้ใช้กดปุ่ม
ยืนยัน

```
Future<void> showConfirmDialog(  
  BuildContext context,  
  Function onConfirm,  
) async {  
  return showDialog<void>(  
    context: context,  
    barrierDismissible: false,  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('ยืนยันการทำการรายการ'),  
        content: const Text('คุณต้องการลบข้อมูลนี้ใช่หรือไม่?'),  
        actions: <Widget>[  
          TextButton(  
            child: const Text('ยกเลิก'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ), // TextButton  
          TextButton(  
            child: const Text('ยืนยัน', style: TextStyle(color: Colors.red)),  
            onPressed: () {  
              Navigator.of(context).pop();  
              onConfirm();  
            },  
          ), // TextButton  
        ], // <Widget>[]  
      ); // AlertDialog  
    },  
  );  
}
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

ปิด dialog เมื่อผู้ใช้กดปุ่ม ยกเลิก หรือ ยืนยัน

```
Future<void> showConfirmDialog(  
  BuildContext context,  
  Function onConfirm,  
) async {  
  return showDialog<void>(  
    context: context,  
    barrierDismissible: false,  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('ยืนยันการทำการรายการ'),  
        content: const Text('คุณต้องการลบข้อมูลนี้ใช่หรือไม่?'),  
        actions: <Widget>[  
          TextButton(  
            child: const Text('ยกเลิก'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ), // TextButton  
          TextButton(  
            child: const Text('ยืนยัน', style: TextStyle(color: Colors.red)),  
            onPressed: () {  
              Navigator.of(context).pop();  
              onConfirm();  
            },  
          ), // TextButton  
        ], // <Widget>[]  
      ); // AlertDialog  
    },  
  );  
}
```

แก้ไข ActivityScreen

/lib/screens/activity_screen.dart

เรียก callback

```
Future<void> showConfirmDialog(
  BuildContext context,
  Function onConfirm,
) async {
  return showDialog<void>(
    context: context,
    barrierDismissible: false,
    builder: (BuildContext context) {
      return AlertDialog(
        title: const Text('ยืนยันการทำการรายการ'),
        content: const Text('คุณต้องการลบข้อมูลนี้ใช่หรือไม่?'),
        actions: <Widget>[
          TextButton(
            child: const Text('ยกเลิก'),
            onPressed: () {
              Navigator.of(context).pop();
            },
          ), // TextButton
          TextButton(
            child: const Text('ยืนยัน', style: TextStyle(color: Colors.red)),
            onPressed: () {
              Navigator.of(context).pop();
              onConfirm();
            },
          ), // TextButton
        ], // <Widget>[]
      ); // AlertDialog
    },
  );
}
```

แก้ไข ActivityScreen

[/lib/screens/activity_screen.dart](#)

แก้ไขการทำงานของปุ่มลบ
ทดสอบโปรแกรม

```
onPressed: () {  
  showConfirmDialog(context, () async {  
    var (isError, errorMessage) = await _doDeleteActivity();  
  
    if (!isError) {  
      Navigator.pop(context);  
    } else {  
      showDialog(  
        context: context,  
        builder: (context) {  
          return AlertDialog(content: Text(errorMessage));  
        },  
      );  
    }  
  });  
},  
// IconButton
```